Endpoint-Proximal Stochastic Steering for Robotic Manipulation with Flow-Matching Policies

Chunxiao Lu, Yongyao Li, Jun Li, Yufei Liu, and Weihua Li, *Member, IEEE*

***Abstract*—Flow-based visuomotor policies have emerged as a promising paradigm for modeling multimodal robot action distributions. However, real-world deployments often introduce task-specific constraints and user preferences that are not considered during training, while collecting additional demonstrations or fine-tuning pretrained policies can be costly. To address this need, we propose Endpoint-Proximal Stochastic Steering (EPSS), a training-free method for guiding pretrained flow-based visuomotor policies at inference time. At each sampling step, EPSS estimates the clean action and noise endpoints from the pretrained policy, enabling guidance to be applied directly in the predicted action space. Building on this representation, an adaptive proximal update based on the Huber loss balances alignment with target actions and the pretrained policy's predictions. Furthermore, pCN re-noising refreshes the noise endpoint to support exploration of alternative action candidates. We evaluate EPSS on a range of challenging simulated and real-world tasks. Experimental results demonstrate that EPSS consistently outperforms existing inference-time steering methods and improves task success rates across multiple tasks while alignment with target actions. Videos and code can be found on the project website: https://lucxdd.github.io/epss-web/**

***Index Terms*—Flow matching, imitation learning, inference-time steering, robotic manipulation.**

I. Introduction

THE rise of generative visuomotor policies, including diffusion models [1], [2] and flow matching models [3], [4], has established generative modeling as a prominent paradigm for a wide range of complex robotic manipulation tasks. These policies can model high-dimensional, multi-modal distributions robotic behaviors through learning from diverse robot demonstrations [5], [6]. This capability is particularly valuable in multi-modal manipulations where the same observation and task objective may allow for different yet equally valid trajectories, grasp configurations, and interaction strategies.

Despite these advances, adapting pretrained flow-based policies to changing requirements in real-world robotic tasks remains challenging. The actions generated by these policies are largely determined by the behaviors learned during training [7], while practical applications may involve new task constraints or user preferences that are not covered by the training data. A straightforward solution is to collect additional demonstrations or fine-tune the policy to achieve the desired behavior [8], [9]. However, these approaches need additional data collection and computational costs, limiting their flexibility in dynamic deployment scenarios. Online reinforcement learning offers another option, but requires further interactions with physical robots and may introduce safety risks [10]. These limitations motivate the development of inference-time guidance methods that can steer generated actions toward new task objectives without updating the pretrained policy, while remaining consistent with its learned action prior.

Recent studies have explored inference-time steering methods that modify the sampling process of pretrained generative policies [11], [12]. These methods use various forms of guidance, including human feedback, target policy actions, value estimates, and task-specific constraints, to guide action generation [8], [13], [14]. However, satisfying external guidance objectives does not always lead to reliable robot actions. In particular, directly applying guidance gradients during sampling may push generated actions away from the learned action distribution, especially when the guidance is inaccurate or conflicts with the policy's learned behavior [14], [15]. Such deviations can lead to abrupt motions, constraint violations, or task failures during execution. These challenges highlight the need to balance external guidance with the pretrained policy's learned behavior while adapting to different guidance conditions.

To address this challenge, we propose Endpoint-Proximal Stochastic Steering (EPSS), a training-free framework for guiding pretrained flow-matching policies at inference time. The key idea is to separate action correction from stochastic refinement by estimating the action and noise endpoints of the flow process. EPSS first uses a biased initialization to incorporate target information into sampling. During flow refinement, an adaptive proximal update with a Huber loss

Manuscript received Month xx, 2xxx; revised Month xx, xxxx; accepted Month x, xxxx. This work was supported in part by the National Natural Science Foundation of China (Grant Nos. 52175007, 52595711, 52502524, and 52425502), in part by the Shandong Provincial Natural Science Foundation (Grant Nos. ZR2024YQ035 and ZR2024QE256), and in part by the Taishan Scholars Program (Grant No. tsqn202507137).

Chunxiao Lu, Jun Li, and Weihua Li are with School of Mechatronics Engineering, Harbin Institute of Technology, Harbin 150001, China (email: 25b308052@stu.hit.edu.cn; junli@hit.edu.cn; liweihua@hit.edu.cn)

Yongyao Li and Yufei Liu are with Component Research and Development Department, Humanoid Robot (Shanghai) Co., Ltd. , Shanghai, 200000, China (email: yongyao.li@openloong.net; liuyufei@openloong.net)

adjusts the predicted action endpoint toward the reference action while accounting for the original policy prediction. Meanwhile, a preconditioned Crank–Nicolson (pCN) update refreshes the noise endpoint to support the exploration of alternative actions. The updated endpoints are then used to continue flow sampling. Through these two updates, EPSS enables the policy to respond to external guidance while maintaining the influence of its learned behavior. The proposed method is evaluated on multimodal simulation benchmarks and real-world robotic manipulation tasks. Experimental results demonstrate that EPSS improves alignment with external references while maintaining task performance under different guidance conditions.

The main contributions of this paper are summarized as follows:

- We propose EPSS, a training-free steering framework for pretrained flow-matching policies. By estimating the action and noise endpoints, EPSS provides a direct way to guide action generation without modifying the pretrained policy.
- We develop an adaptive proximal guidance mechanism based on the Huber loss to balance external references and pretrained action predictions. This mechanism is combined with pCN re-noising to support stochastic refinement and biased initialization to incorporate target information early in sampling.
- We conduct extensive experiments on multimodal simulation benchmarks and real-world robotic manipulation tasks. The results demonstrate the effectiveness of EPSS in improving reference alignment while maintaining task execution performance.

II. Related Work

A. Generative Robot Policies

Generative visuomotor policies have shown strong capabilities in learning complex robot behaviors from demonstrations. Diffusion-based policies generate action sequences through iterative denoising and support closed-loop execution using receding-horizon control [1], [16]. Further studies incorporate 3D visual representations to improve generalization across different spatial configurations [17]. Flow matching provides another approach to action generation by learning a time-dependent velocity field that transforms noise into actions [18]. FlowPolicy [19] improves inference efficiency through consistency flow matching. These advances provide a strong foundation for learning diverse robot behaviors and further adapting generated actions during deployment.

B. Inference-Time Policy Steering

Inference-time policy steering aims to adapt generated actions to external objectives using pretrained generative models. In shared autonomy, To the Noise and Back [20] combines user inputs with learned policy behaviors through a noising and denoising process. ITPS [14] explores different guidance strategies, including candidate ranking, gradient guidance, and stochastic sampling. Its results highlight the trade-off between alignment with user intent and consistency with the learned policy distribution.

Recent work has also investigated methods for steering robot actions using flow-based models. Flow Reversal Steering [13] maps coarse reference actions into the noise space through reverse flow integration and generates refined actions using the pretrained policy. GLOVES [21] uses a learned expert flow to identify and correct proposed actions that deviate from expert behavior. Other methods use additional models to provide guidance during sampling. DynaGuide [9] employs a learned dynamics model to guide diffusion policies toward desired visual outcomes. These methods offer different ways to control generated actions, but their guidance mechanisms depend on additional learned models.

Related advances in flow-based inverse problems offer another perspective on guiding generative sampling. FlowDPS and FlowLPS [22], [23] decompose intermediate flow states into estimated clean and noise endpoints, combining target-consistency corrections with stochastic updates. These methods focus on satisfying measurement constraints in inverse problems. In robotic manipulation, external action references may be inaccurate or inconsistent with the behavior learned by the policy. EPSS addresses this setting through adaptive proximal guidance with a Huber loss in the predicted action endpoint space, along with stochastic noise refinement. This design allows the policy to respond to external action references while accounting for its original action predictions.

III. Preliminaries

A. Conditional Flow Matching

Flow matching learns a velocity field that transforms samples from a source distribution into a target data distribution [24]. For robotic manipulation, let c denote the observation condition and $\mathbf{a} \in \mathbb{R}^D$ an expert action sequence, where $D = Hd$, with action horizon H and per-step action dimension d. Following rectified flow [25], we adopt a straight interpolation between an expert action $\mathbf{a} \sim p_{\text{data}}(\cdot \mid c)$ and Gaussian noise $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_D)$, given by

$$\mathbf{a}_t = (1-t)\mathbf{z} + t\mathbf{a}, \qquad \frac{d\mathbf{a}_t}{dt} = \mathbf{a} - \mathbf{z}. \tag{1}$$

Here, $t \in [0,1]$ and each endpoint pair defines a straight path with constant target velocity. The conditional velocity field $\mathbf{v}_\theta$ is trained by minimizing

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\mathbf{a},\mathbf{z},t,c}\left[\|\mathbf{v}_\theta(\mathbf{a}_t, t, c) - (\mathbf{a}-\mathbf{z})\|_2^2\right], \tag{2}$$

where $t \sim \mathcal{U}[0,1]$.

At inference time, an initial state $\mathbf{a}_0 = \mathbf{z}$ is sampled from the standard Gaussian distribution. The action sequence is generated by solving the learned flow ODE

$$\frac{d\mathbf{a}_t}{dt} = \mathbf{v}_\theta(\mathbf{a}_t, t, c). \tag{3}$$

Using N Euler steps with $t_i = i/N$ and $\Delta t = 1/N$, the sampling update is

$$\mathbf{a}_{t_{i+1}} = \mathbf{a}_{t_i} + \Delta t\, \mathbf{v}_\theta(\mathbf{a}_{t_i}, t_i, c). \tag{4}$$

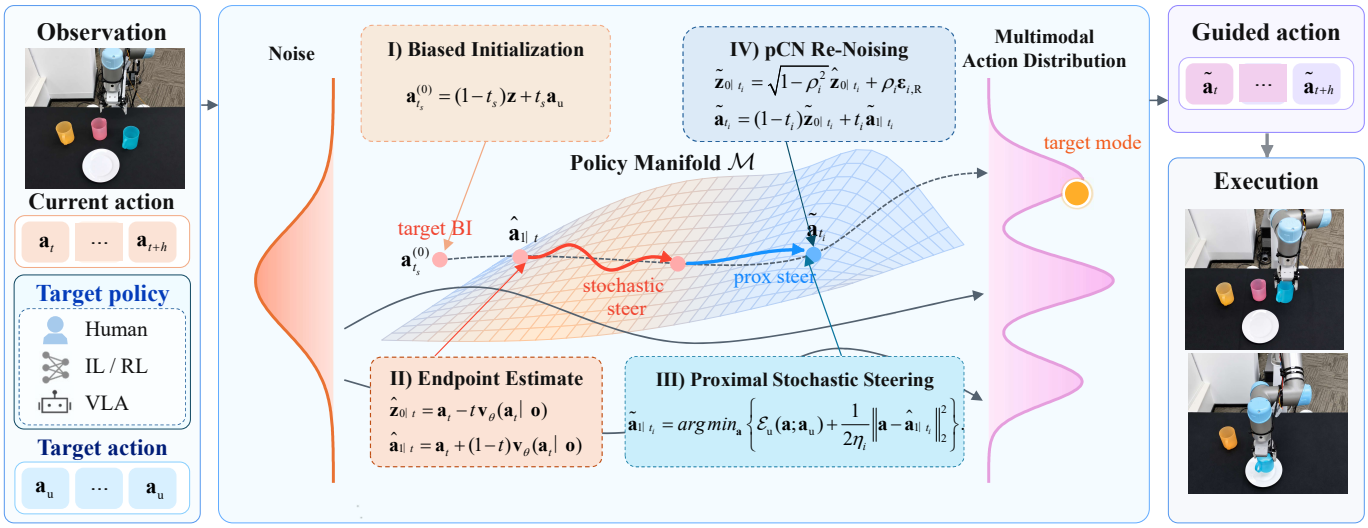


Fig. 1. Overview of EPSS. Given an external action reference, EPSS guides the sampling process of a pretrained flow policy to generate an action sequence for robot execution.

The final state $\mathbf{a}_1$ gives the generated action sequence. EPSS guides this sampling process through estimated action and noise endpoints while keeping the pretrained policy frozen.

IV. Method

Given the current observation $\mathbf{o}$, we consider a pretrained flow-matching policy $\mathbf{v}_\theta$ and a target action sequence $\mathbf{a}_\mathrm{u}$ provided by a user or an external policy. Let c denote the encoded observation. The goal of EPSS is to guide action generation toward the target while preserving the behavior learned by the pretrained policy. The policy parameters remain frozen throughout the process.

As illustrated in Fig. 1, EPSS begins by incorporating the target action into a partially noised initial state. At each sampling step, the pretrained flow policy predicts a velocity field, which is used to estimate the clean action and noise endpoints. The predicted action endpoint is then refined toward the target through an adaptive proximal update, while the noise endpoint is refreshed to support stochastic exploration. The updated endpoints are recombined to reconstruct the flow state, allowing the pretrained policy to continue the sampling process. By repeating these steps, EPSS guides action generation toward the target while preserving the influence of the pretrained policy. Fig. 2 compares the sampling mechanisms of EPSS and representative inference-time steering methods.

A. Partial Biased Initialization

Standard flow matching generates actions by sampling Gaussian noise $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_D)$ and transporting it through the learned velocity field. The initial noise determines the starting point of the generative process and influences the resulting action sequence. To incorporate external guidance from the beginning of sampling, we initialize the flow using a mixture of Gaussian noise and the target action.

Specifically, we start from an intermediate flow time t_s and construct the initial state as

$$\mathbf{a}_{t_s}^{(0)} = (1 - t_s)\mathbf{z} + t_s \mathbf{a}_\mathrm{u}, \qquad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_D). \tag{5}$$

Here, $t_s \in [0, 1)$ controls the contribution of the target action to the initial state. When $t_s = 0$, the initialization reduces to standard Gaussian sampling. As t_s increases, the initial state becomes closer to the target action, providing a stronger reference bias. At the same time, the remaining flow interval becomes shorter.

This initialization encourages sampling toward the desired behavior while retaining a random component. The subsequent flow steps further refine the generated action using the pretrained policy and the external reference.

B. Flow Endpoint Decomposition

External guidance is defined over clean action sequences, while the intermediate flow state contains both action and noise components. To apply guidance directly in action space, we estimate the clean action endpoint from the current flow state and the velocity predicted by the pretrained policy. We also estimate the corresponding noise endpoint for stochastic refinement.

Following the linear probability path introduced in Section III-A, we have

$$\mathbf{a}_t = (1 - t)\mathbf{z} + t\mathbf{a}_1, \qquad t \in [0, 1], \tag{6}$$

where $\mathbf{z}$ and $\mathbf{a}_1$ denote the noise and action endpoints, respectively. The conditional velocity along this path is $\mathbf{a}_1 - \mathbf{z}$. Under the flow matching objective, the corresponding marginal velocity is

$$\mathbf{v}_t(\mathbf{a}_t \mid c) = \mathbb{E}[\mathbf{a}_1 - \mathbf{z} \mid \mathbf{a}_t, c]. \tag{7}$$

The relation between the intermediate state, velocity, and endpoint estimates can be derived for a general affine probability path.

a) Proposition 1 (Endpoint Decomposition).: Consider a differentiable affine probability path

$$\mathbf{a}_t = \sigma_t \mathbf{z} + \beta_t \mathbf{a}_1, \tag{8}$$

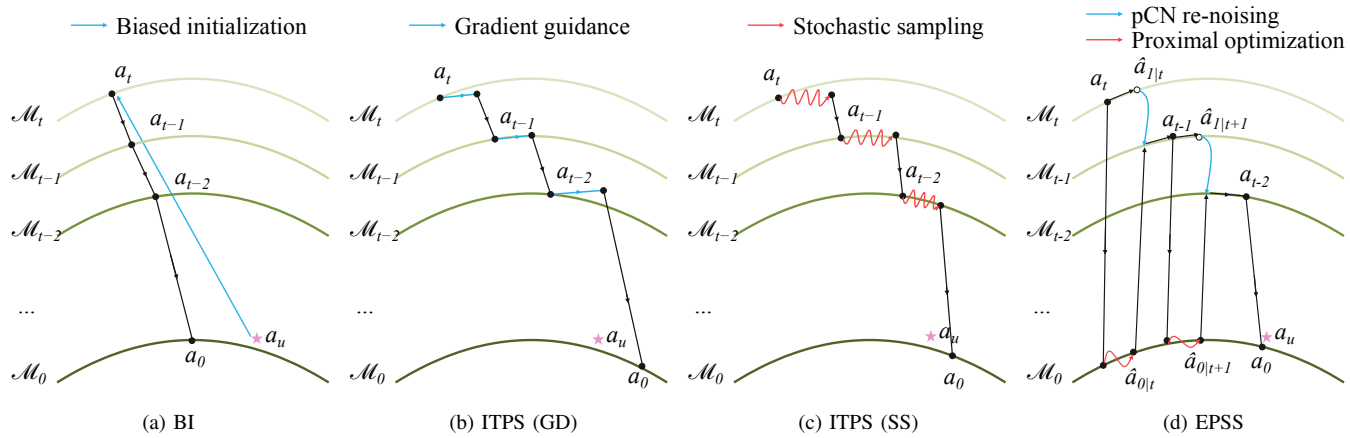


Fig. 2. Comparison of inference-time policy steering methods. (a) BI biases initialization, but its guidance may weaken during sampling. (b) ITPS (GD) applies guidance gradients that may move actions away from the learned distribution. (c) ITPS (SS) uses repeated Langevin updates at additional computational cost. (d) EPSS combines proximal action refinement and pCN re-noising to guide sampling while retaining the pretrained policy's influence.

where σ_t and β_t are differentiable scalar functions of time. The marginal velocity is given by

$$\begin{aligned}\mathbf{v}_t(\mathbf{a}_t \mid c) = \dot{\sigma}_t \mathbb{E}[\mathbf{z} \mid \mathbf{a}_t, c] \\ + \dot{\beta}_t \mathbb{E}[\mathbf{a}_1 \mid \mathbf{a}_t, c].\end{aligned} \tag{9}$$

Define $\Delta_t = \sigma_t \dot{\beta}_t - \dot{\sigma}_t \beta_t$. For $\Delta_t \neq 0$, the conditional endpoint means are

$$\mathbb{E}[\mathbf{z} \mid \mathbf{a}_t, c] = \frac{\dot{\beta}_t \mathbf{a}_t - \beta_t \mathbf{v}_t(\mathbf{a}_t \mid c)}{\Delta_t}, \tag{10}$$

$$\mathbb{E}[\mathbf{a}_1 \mid \mathbf{a}_t, c] = \frac{\sigma_t \mathbf{v}_t(\mathbf{a}_t \mid c) - \dot{\sigma}_t \mathbf{a}_t}{\Delta_t}. \tag{11}$$

Proof. Taking the conditional expectation of (8) and its time derivative gives two linear equations for the conditional endpoint means. The determinant of the resulting system is Δ_t. Solving this system yields (10) and (11). □

For the linear path in (6), we have $\sigma_t = 1 - t$, $\beta_t = t$, and $\Delta_t = 1$. Using the learned velocity field $\mathbf{v}_\theta$ to estimate the marginal velocity gives

$$\hat{\mathbf{z}}_{0|t} = \mathbf{a}_t - t\mathbf{v}_\theta(\mathbf{a}_t, t, c), \tag{12}$$

$$\hat{\mathbf{a}}_{1|t} = \mathbf{a}_t + (1 - t)\mathbf{v}_\theta(\mathbf{a}_t, t, c). \tag{13}$$

Here, $\hat{\mathbf{z}}_{0|t}$ and $\hat{\mathbf{a}}_{1|t}$ are the estimated noise and clean action endpoints. With the exact marginal velocity, these expressions recover the conditional endpoint means of the linear probability path.

The two estimates satisfy the following reconstruction identity:

$$\mathbf{a}_t = (1 - t)\hat{\mathbf{z}}_{0|t} + t\hat{\mathbf{a}}_{1|t}. \tag{14}$$

This decomposition provides two variables for guiding the sampling process. The estimated action endpoint allows the external reference to be applied directly in action space. The estimated noise endpoint can be updated separately to introduce stochastic variation during sampling.

C. pCN Re-Noising

Flow endpoint decomposition provides a noise estimate that can be updated during generation. To support the exploration of alternative actions, EPSS introduces new randomness through the noise endpoint while retaining information from the current sample. We adopt a correlated Gaussian update based on preconditioned Crank–Nicolson (pCN) sampling [26].

At flow time t_i, the noise endpoint is updated as

$$\begin{aligned}\tilde{\mathbf{z}}_{0|t_i} = \sqrt{1 - \rho_i^2}\, \hat{\mathbf{z}}_{0|t_i} + \rho_i \boldsymbol{\varepsilon}_{i,\mathrm{R}}, \\ \boldsymbol{\varepsilon}_{i,\mathrm{R}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_D),\end{aligned} \tag{15}$$

where $\rho_i \in [0, 1]$ controls the amount of newly introduced noise. When $\rho_i = 0$, the current noise estimate is preserved. As ρ_i increases, the contribution of fresh Gaussian noise becomes larger. At $\rho_i = 1$, the noise endpoint is replaced by a new Gaussian sample.

The pCN update maintains a controllable connection to the current noise estimate while introducing new sampling directions. The refreshed noise endpoint is subsequently combined with the refined action endpoint to construct the next flow state. Through the linear flow interpolation, the contribution of the noise endpoint gradually decreases as t approaches 1, allowing the final stages of sampling to focus on the generated action.

D. Adaptive Proximal Stochastic Steering

The estimated clean action endpoint provides a direct representation for applying external guidance. However, a reference action may differ from the behavior predicted by the pretrained policy. To control the influence of the reference, EPSS uses proximal optimization to refine the predicted action endpoint while preserving information from the original prediction.

We first define a guidance energy based on the Huber loss. At sampling time t_i, the guidance energy is

$$\mathcal{E}_{\mathrm{u}}(\mathbf{a}; \mathbf{a}_{\mathrm{u}}) = \sum_{j=1}^{D} h_\delta(a_j - a_{\mathrm{u},j}), \tag{16}$$

where D is the action dimensionality and h_δ is defined as

$$h_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \le \delta, \\ \delta\left(|r| - \frac{\delta}{2}\right), & |r| > \delta. \end{cases} \tag{17}$$

The parameter $\delta > 0$ determines the transition between the quadratic and linear regions. For small discrepancies, the quadratic penalty encourages close alignment with the reference. For larger discrepancies, the linear penalty limits the influence of large reference errors. This allows the guidance objective to account for coarse or inaccurate target actions.

We refine the predicted clean action endpoint using the proximal operator [27]

$$\begin{aligned} \tilde{\mathbf{a}}_{1|t_i} &= \mathrm{prox}_{\eta_i \mathcal{E}_\mathrm{u}}\left(\hat{\mathbf{a}}_{1|t_i}\right) \\ &= \arg\min_{\mathbf{a}} \left\{ \mathcal{E}_\mathrm{u}(\mathbf{a}; \mathbf{a}_\mathrm{u}) + \frac{1}{2\eta_i} \left\| \mathbf{a} - \hat{\mathbf{a}}_{1|t_i} \right\|_2^2 \right\}, \end{aligned} \tag{18}$$

where $\eta_i > 0$ controls the strength of the guidance correction. The Huber energy encourages alignment with the target action, while the quadratic term keeps the refined endpoint close to the pretrained prediction.

Since the guidance energy is separable across action dimensions, the proximal optimization admits an elementwise closed-form solution. For each dimension j, we obtain

$$\tilde{a}_{1|t_i,j} = \hat{a}_{1|t_i,j} - \eta_i \,\mathrm{clip}\left(\frac{\hat{a}_{1|t_i,j} - a_{\mathrm{u},j}}{1+\eta_i}, -\delta, \delta \right), \tag{19}$$

where $\mathrm{clip}(x, l, u)$ restricts x to the interval $[l, u]$. This solution gives a direct action correction without iterative optimization. For large reference errors, the magnitude of the correction in each action dimension is bounded by $\eta_i \delta$.

To keep generated actions close to the pretrained policy's learned action manifold, we gradually reduce the influence of external guidance as sampling progresses. Specifically, we keep the model weight fixed and decrease the guidance weight over flow time:

$$c_i^{\mathrm{model}} = c_0^{\mathrm{model}}, \tag{20}$$

$$c_i^{\mathrm{u}} = c_0^{\mathrm{u}} \exp(-\kappa t_i), \tag{21}$$

where $c_0^{\mathrm{model}} > 0$ and $c_0^{\mathrm{u}} > 0$ are the initial weights, and $\kappa \ge 0$ controls the decay rate. This allows the external reference to guide action generation in the early stages of sampling, while giving the pretrained policy greater influence over the final action.

The corresponding proximal parameter is defined by the ratio of the guidance and model weights:

$$\eta_i = \frac{c_i^{\mathrm{u}}}{c_i^{\mathrm{model}}}. \tag{22}$$

This schedule provides stronger reference corrections early in sampling and gradually reduces the correction strength as the generated action approaches the final endpoint. When $\kappa = 0$, the proximal parameter remains constant throughout sampling.

After proximal refinement, the updated action endpoint and the noise endpoint obtained from (15) are combined at the current flow time:

$$\tilde{\mathbf{a}}_{t_i} = (1 - t_i)\tilde{\mathbf{z}}_{0|t_i} + t_i \tilde{\mathbf{a}}_{1|t_i}. \tag{23}$$

Algorithm 1: Endpoint-Proximal Stochastic Steering (EPSS)

Input: Pretrained policy $\mathbf{v}_\theta$, observation c, reference action $\mathbf{a}_\mathrm{u}$, start time t_s, steps N
Output: Guided action sequence $\tilde{\mathbf{a}}$

▷ *Biased initialization*
$t_i \leftarrow t_s + (1 - t_s)i/N,\ i = 0, \ldots, N$
Sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_D)$
$\mathbf{a}_{t_0} \leftarrow (1 - t_0)\mathbf{z} + t_0 \mathbf{a}_\mathrm{u}$
for $i = 0, \ldots, N-1$ **do**
  ▷ *Estimate action and noise endpoints*
  $\mathbf{v}_i \leftarrow \mathbf{v}_\theta(\mathbf{a}_{t_i}, t_i, c)$
  $\hat{\mathbf{z}}_{0|t_i} \leftarrow \mathbf{a}_{t_i} - t_i \mathbf{v}_i$
  $\hat{\mathbf{a}}_{1|t_i} \leftarrow \mathbf{a}_{t_i} + (1 - t_i)\mathbf{v}_i$
  ▷ *Update guidance weight*
  $c_i^{\mathrm{model}} \leftarrow c_0^{\mathrm{model}}$
  $c_i^{\mathrm{u}} \leftarrow c_0^{\mathrm{u}} e^{-\kappa t_i}$
  $\eta_i \leftarrow c_i^{\mathrm{u}} / c_i^{\mathrm{model}}$
  ▷ *Huber proximal correction*
  $\tilde{\mathbf{a}}_{1|t_i} \leftarrow \mathrm{prox}_{\eta_i \mathcal{E}_\mathrm{u}}(\hat{\mathbf{a}}_{1|t_i})$
  ▷ *pCN noise refresh*
  Sample $\boldsymbol{\varepsilon}_{i,\mathrm{R}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_D)$
  $\tilde{\mathbf{z}}_{0|t_i} \leftarrow \sqrt{1 - \rho_i^2}\, \hat{\mathbf{z}}_{0|t_i} + \rho_i \boldsymbol{\varepsilon}_{i,\mathrm{R}}$
  ▷ *Reconstruct and advance the flow*
  $\tilde{\mathbf{a}}_{t_i} \leftarrow (1 - t_i)\tilde{\mathbf{z}}_{0|t_i} + t_i \tilde{\mathbf{a}}_{1|t_i}$
  $\mathbf{a}_{t_{i+1}} \leftarrow \tilde{\mathbf{a}}_{t_i} + (t_{i+1} - t_i)\mathbf{v}_\theta(\tilde{\mathbf{a}}_{t_i}, t_i, c)$
$\tilde{\mathbf{a}} \leftarrow \mathbf{a}_{t_N}$
return $\tilde{\mathbf{a}}$

The reconstructed state contains both the reference-guided action estimate and the refreshed noise. The pretrained velocity field is then evaluated at this state to advance the flow:

$$\mathbf{a}_{t_{i+1}} = \tilde{\mathbf{a}}_{t_i} + (t_{i+1} - t_i)\mathbf{v}_\theta(\tilde{\mathbf{a}}_{t_i}, t_i, c). \tag{24}$$

The proximal correction guides the predicted action toward the reference, and pCN re-noising introduces stochastic variation into subsequent sampling. Recomputing the velocity at the reconstructed state allows both updates to influence the remaining flow trajectory. The procedure continues until $t_N = 1$, yielding the final guided action sequence. Algorithm 1 summarizes the complete EPSS sampling process.

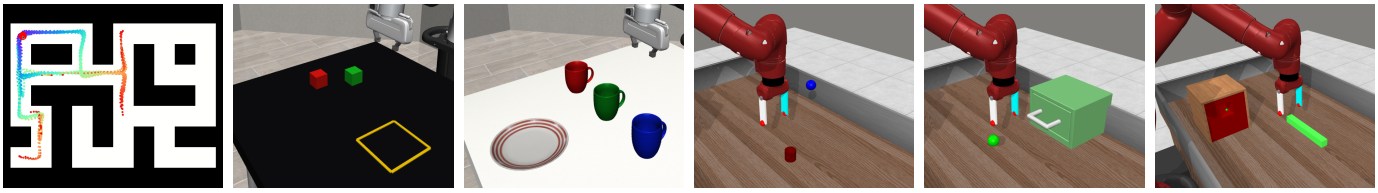
V. Experiments

This section describes the simulation settings and the real-world task setup. We evaluate EPSS and several guidance methods on six simulation tasks and a real-world robot task, covering a range of environmental conditions and control modes.

A. Experimental Setup

a) Environments and Metrics: We use Maze2D, Push Block, and Cup Place to evaluate reference-guided action generation in tasks with multiple possible behaviors. Maze2D is a planar navigation environment based on D4RL [28]. The agent predicts continuous waypoints through free space while avoiding obstacles. We report the average Euclidean distance between generated and reference waypoints (Avg ℓ_2, measured in grid cells) and the collision rate (CR). Lower values indicate closer reference alignment and fewer collisions, respectively.

Push Block requires a robot arm to push the selected block into a target region. Different blocks provide distinct choices



(a) Maze2D (b) Push Block (c) Cup Place (d) Pick Place (e) Drawer Open (f) Peg Insert Side

Fig. 3. Simulation environments and tasks. (a) Maze navigation with obstacles. (b) Pushing a block to a target region. (c) Placing a selected cup on a plate. (d) Picking and placing an object. (e) Opening a drawer. (f) Inserting a peg into a side opening.

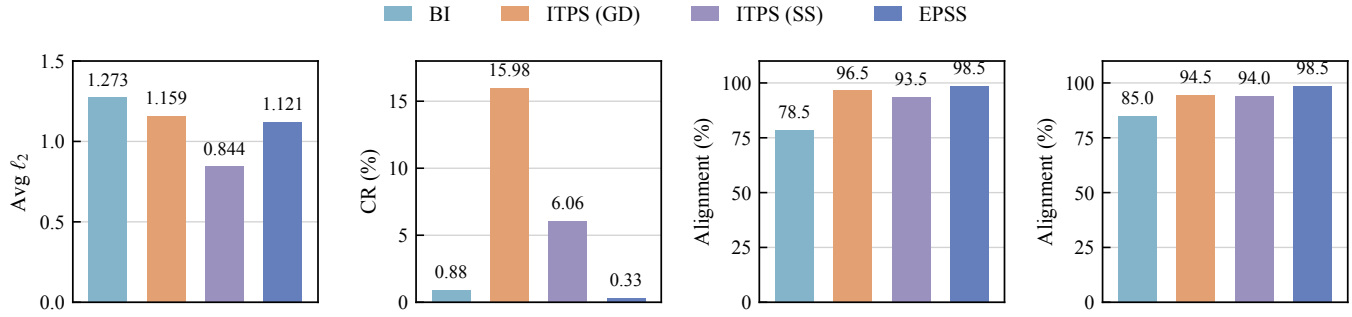


Fig. 4. Reference alignment and collision performance on multimodal simulation tasks. The panels show Avg ℓ_2 and CR (%) on Maze2D, Push Block alignment (%), and Cup Place alignment (%), from left to right. Lower values are better for Avg ℓ_2 and CR, while higher values are better for alignment.

for the external reference. In Cup Place [29], a Franka arm selects one of three cups, grasps it, and places it upright on a plate before releasing the grasp. For both tasks, we report task success and alignment with the target behavior.

We also evaluate three MetaWorld tasks [30], including Pick Place, Drawer Open, and Peg Insert Side. These tasks involve object transport, drawer opening, and precise insertion, respectively. We use task success rate to assess whether guided actions support complete manipulation sequences. The six simulation environments are shown in Fig. 3.

b) Baselines: We compare EPSS with the unguided policy and three inference-time steering methods:

Base Policy [19]. A flow matching policy trained on expert demonstrations generates actions without guidance, providing a reference for evaluating the effects of guidance.

To the Noise and Back: Diffusion for Shared Autonomy (BI) [20]. BI incorporates the reference into the initial sampling state and then follows the pretrained generative process.

Inference-Time Policy Steering through Human Interactions (ITPS (GD)) [14]. This variant applies guidance gradients during sampling to improve alignment with external targets.

Inference-Time Policy Steering through Human Interactions (ITPS (SS)) [14]. Building on gradient guidance, stochastic sampling (SS) performs multiple stochastic sampling updates at the same noise level to balance target alignment with the policy prior.

c) Implementation Details: In the simulation tasks, scripted expert controllers provide demonstrations for training the flow-matching policies. During evaluation, the pretrained policies remain fixed. We use a coarse heuristic controller as the external target policy. It provides reference actions based on the current state, allowing each steering method to guide the same task toward a specified behavior. The policy predicts an action sequence of length 16 and executes the first 8 steps in a receding-horizon manner. Table I summarizes the main training and model settings.

TABLE I
MODEL HYPERPARAMETERS

Component	Value
Visual backbone	ResNet-18
Pretrained weights	ImageNet
Velocity network	Conditional 1D U-Net
U-Net channel widths	$(256, 512, 1024)$
Time embedding dimension	128
Observation history length	2
Prediction horizon	16
Executed action steps	8
Learning rate	1×10^{-4}
Weight decay	1×10^{-6}

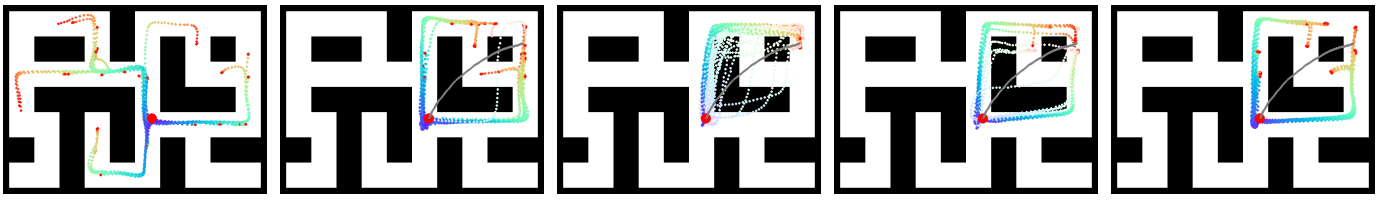
B. Results and Analysis

a) Manipulation performance: Table II presents the performance of different methods across the simulated tasks. EPSS achieves the highest mean success rate in every task. On Push Block and Cup Place, it reaches 91.0% and 88.0%, exceeding the strongest baseline in each task by 6.5 and 5.0 percentage points, respectively. These results show that improved control over the target behavior can also lead to more successful task execution.

The MetaWorld results further demonstrate the effectiveness of EPSS on multistage manipulation tasks. Representative

TABLE II
TASK SUCCESS RATES (%) ON SIMULATION TASKS. RESULTS ARE REPORTED AS THE MEAN AND STANDARD DEVIATION.

Method	Push Block	Cup Place	Pick Place	Drawer Open	Peg Insert Side
Base Policy	78.50 ± 8.83	72.00 ± 7.52	88.00 ± 4.21	95.50 ± 4.97	19.00 ± 6.58
BI	82.50 ± 6.77	79.00 ± 7.00	87.50 ± 5.40	93.00 ± 4.83	60.00 ± 12.24
ITPS (GD)	82.50 ± 5.40	68.50 ± 8.83	85.50 ± 5.99	85.50 ± 5.27	82.00 ± 2.58
ITPS (SS)	84.50 ± 8.31	83.00 ± 7.89	90.00 ± 4.08	86.50 ± 5.80	79.00 ± 4.59
EPSS	$\mathbf{91.00 \pm 6.58}$	$\mathbf{88.00 \pm 4.22}$	$\mathbf{92.50 \pm 3.53}$	$\mathbf{97.00 \pm 4.21}$	$\mathbf{88.50 \pm 4.74}$



(a) Base Policy (b) BI (c) ITPS (GD) (d) ITPS (SS) (e) EPSS

Fig. 5. Generated trajectories in Maze2D. Black regions are obstacles, white regions are free space, colored points are sampled trajectories, and gray curves are reference paths. All methods use the same starting point and reference trajectory.

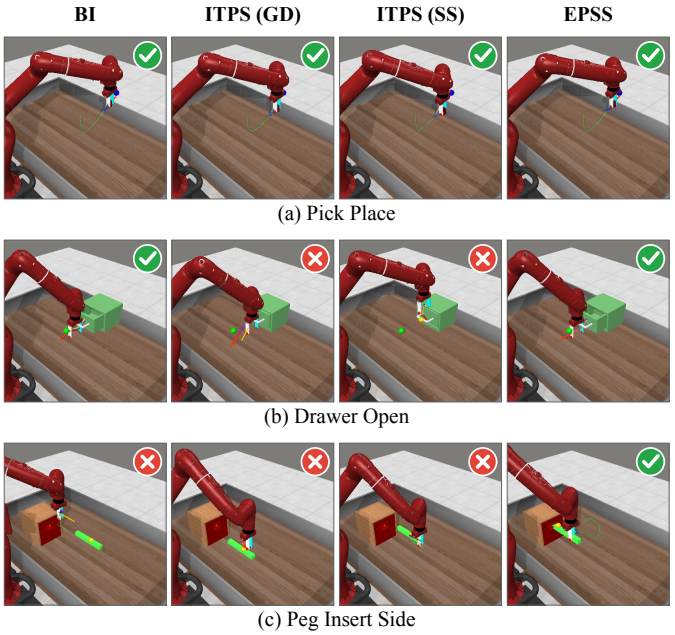
executions are shown in Fig. 6. In Pick Place and Drawer Open, EPSS achieves success rates of 92.5% and 97.0%. The largest improvement over the unguided policy occurs in Peg Insert Side, where the mean success rate increases from 19.0% to 88.5%. This task requires the robot to align the peg and complete a contact-sensitive insertion. The result highlights the value of guiding pretrained action generation toward a task objective while retaining the behavior needed for successful execution.

b) Reference Alignment and Trajectory Feasibility: Fig. 4 compares the methods on Maze2D, Push Block, and Cup Place. These tasks test whether generated actions can follow a target reference while remaining suitable for the environment. In Maze2D, the reference path may pass near or through obstacles, making alignment and collision avoidance competing objectives. The unguided policy follows its learned behavior but has limited control over the requested path. BI introduces reference information at initialization, while ITPS (GD) and ITPS (SS) apply stronger corrections during sampling. EPSS uses endpoint refinement to incorporate the reference while retaining the pretrained policy's influence on action generation.

The trajectory samples in Fig. 5 provide a visual comparison under the same starting point and reference path. They illustrate how the guidance methods respond to the target and how their sampled paths interact with the obstacles. For EPSS, the resulting trajectories reflect the combined effect of reference-guided action updates and continued flow refinement. The alignment results for Push Block and Cup Place in Fig. 4 further evaluate this ability in tasks with different target objects.

C. Ablation Studies and Mechanism Analysis

We use Maze2D to study how the key components affect reference alignment and collision avoidance.



(a) Pick Place

(b) Drawer Open

(c) Peg Insert Side

Fig. 6. A visualization of the result of all methods on the three MetaWorld tasks.

a) Ablation Study on Biased Initialization: We investigate the effect of biased initialization by varying the starting time t_s from 0 to 0.9 with $\delta = 0.3$ fixed. A larger t_s places the initial state closer to the reference but leaves fewer flow steps for refinement. As shown in Fig. 7, the intermediate range around $t_s = 0.5$ to 0.7 achieves improved reference alignment while keeping collision rates relatively low. Starting closer to the target further reduces Avg ℓ_2, but the collision rate rises sharply. These results show the importance of retaining sufficient flow refinement after biased initialization.

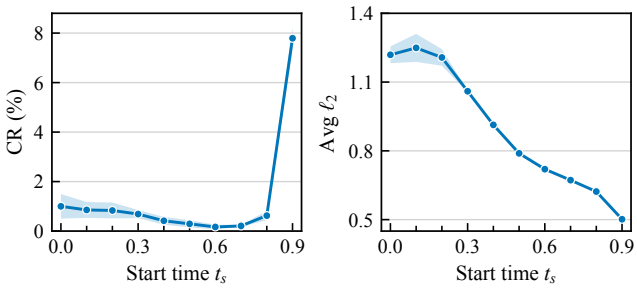


Fig. 7. Sensitivity to the starting time t_s with $\delta = 0.3$. The panels show CR (%) and Avg ℓ_2.

TABLE III
EFFECT OF ADAPTIVE GUIDANCE WEIGHTING ON MAZE2D.

Method	CR (%) ↓	Avg ℓ_2 ↓
w/o Adaptive Weighting	7.15 ± 9.36	0.836 ± 0.250
EPSS	0.33 ± 0.88	1.121 ± 0.315

b) Ablation study on Adaptive Weighting: We examine the effect of decreasing guidance strength over flow time by removing the time-dependent weighting schedule. Table III shows that the constant-weight variant produces a lower Avg ℓ_2 of 0.836, but its mean CR increases to 7.15%. With time-dependent weighting, EPSS achieves a CR of 0.33% and an Avg ℓ_2 of 1.121. Reducing the guidance influence during later sampling steps allows the pretrained policy to contribute more strongly to the final trajectory. This is especially valuable when following the reference too closely would lead to obstacle collisions.

c) Ablation study on pCN Re-Noising: We investigate the role of stochastic noise refinement by disabling pCN re-noising and testing different refresh schedules. Table IV shows that every tested refresh setting reduces both mean CR and Avg ℓ_2 compared with the variant without re-noising. Fixed $\rho = 0.4$ gives the lowest Avg ℓ_2 of 1.086, while $\rho = 0.6$ and $\rho = \sqrt{t}$ both achieve the lowest reported mean CR of 0.33%. These results show that refreshing the noise endpoint improves the search for feasible action trajectories, with the refresh strength controlling the resulting balance between alignment and collision avoidance.

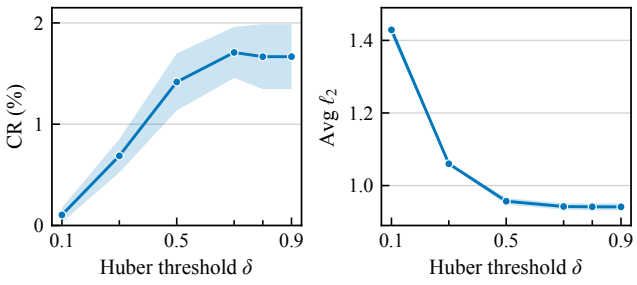


Fig. 8. Sensitivity to the Huber threshold δ with $t_s = 0.3$. The panels show CR (%) and Avg ℓ_2.

TABLE IV
EFFECT OF PCN RE-NOISING ON MAZE2D.

Noise refresh ρ	CR (%) ↓	Avg ℓ_2 ↓
w/o pCN Re-Noising	0.69 ± 1.34	1.142 ± 0.326
0.4	0.58 ± 1.38	1.086 ± 0.299
0.6	0.33 ± 1.06	1.109 ± 0.313
t	0.38 ± 0.86	1.094 ± 0.302
$\sqrt{t}$	0.33 ± 0.88	1.121 ± 0.315

TABLE V
REAL-WORLD CUP PLACE RESULTS.

Method	Success ↑	Alignment (%) ↑
Base Policy (w/o ours)	40/60 (66.7%)	–
BI	47/60 (78.3%)	70.00
EPSS (w/ ours)	**52/60 (86.7%)**	**95.00**

d) The sensitivity of the Huber Threshold: We evaluate the Huber threshold by varying δ from 0.1 to 0.9 with $t_s = 0.3$ fixed. Fig. 8 shows that increasing δ generally improves reference alignment, as reflected by a lower Avg ℓ_2. At the same time, collision rates increase as the proximal update allows larger corrections toward the reference. The improvement in alignment becomes smaller beyond $\delta = 0.5$. A moderate threshold therefore provides a useful balance between following the target and avoiding obstacles.

The ablations show how the components contribute to the same objective. Biased initialization and Huber guidance improve control over the target action, while the adaptive guidance schedule and pCN updates help maintain feasible trajectory generation.

D. Real-World Experiments

To further evaluate our method, we conduct a real-world Cup Place task. An experienced teleoperator collected 60 expert demonstration trajectories using Meta Quest 3 virtual reality controllers. These demonstrations were used to train a flow-matching policy, which remains fixed during evaluation. The task requires the robot to grasp a selected cup and place it on the target plate. We use coarse reference actions to specify the target cup during inference.

Fig. 9 compares action predictions from the same scene before and after applying EPSS. For each target cup, we first obtain an unguided prediction and then generate an action sequence using the external reference. The original predictions are shown only for comparison, while the EPSS-guided actions are executed on the robot. The examples illustrate how inference-time guidance changes the predicted motion toward the selected target and supports grasping and placement.

For quantitative evaluation, each method is independently tested over 60 real-world episodes. As shown in Table V, EPSS achieves a higher task success rate than both the unguided policy and BI, while also improving target alignment over BI. These results demonstrate that reference-guided sampling improves target selection and task execution on a physical robot without modifying the pretrained flow policy.

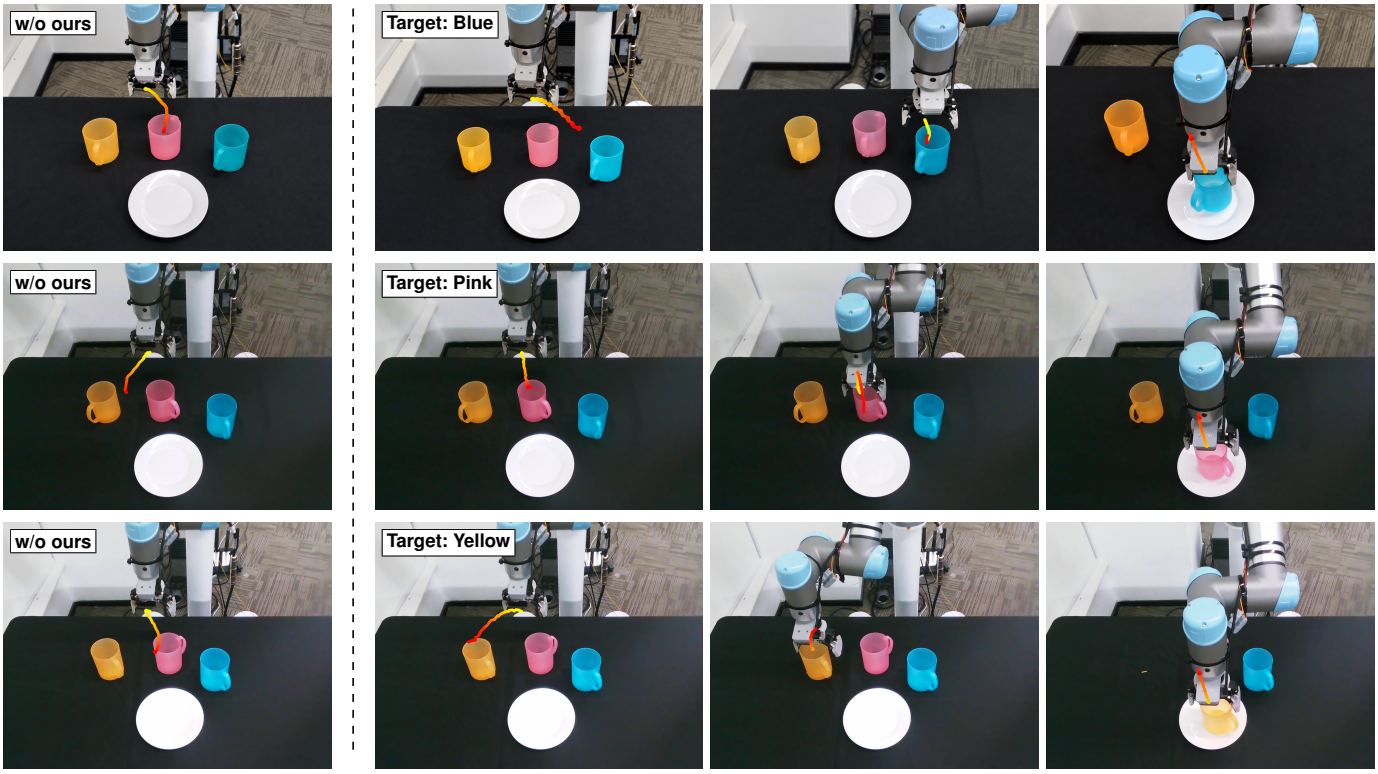


Fig. 9. Real-world Cup Place results for blue, pink, and yellow targets. Each row shows the unguided action prediction, the EPSS-guided prediction, grasping, and placement. The unguided predictions shown here are visualized without execution.

VI. Conclusion

In this paper, we proposed Endpoint-Proximal Stochastic Steering (EPSS), a training-free method for guiding pretrained flow-matching robot policies at inference time. EPSS introduces external reference information through partial biased initialization and estimates action and noise endpoints during flow sampling. An adaptive Huber proximal update guides the predicted action toward the reference, while pCN re-noising supports the exploration of alternative action sequences. The time-dependent guidance schedule gradually reduces external corrections to give the pretrained policy greater influence over the final action. Experiments on simulation benchmarks and real-world robotic tasks demonstrate the effectiveness of EPSS in improving target alignment and task performance without additional retraining. The additional velocity evaluations introduce computational overhead during inference. Future work will focus on improving sampling efficiency and extending EPSS to longer action horizons and multiple guidance sources.

References

[1] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.

[2] B. Zhou, R. Jiao, Y. Li, X. Yuan, F. Fang, and S. Li, "Admittance visuomotor policy learning for general-purpose contact-rich manipulations," *IEEE Transactions on Industrial Electronics*, vol. 72, DOI 10.1109/TIE.2025.3561816, no. 11, pp. 11 520–11 529, 2025.

[3] S. Wang, L. Wang, S. Zhou, J. Tian, J. Li, H. Sun, and W. Tang, "Flowram: Grounding flow matching policy with region-aware mamba framework for robotic manipulation," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, DOI 10.1109/CVPR52734.2025.01137, pp. 12 176–12 186, 2025.

[4] A. Sochopoulos, N. Malkin, N. Tsagkas, J. Moura, M. Gienger, and S. Vijayakumar, "Fast flow-based visuomotor policies via conditional optimal transport couplings," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Lim, S. Song, and H.-W. Park, Eds., vol. 305, pp. 3357–3377. PMLR, 27–30 Sep 2025.

[5] M. Reuss, M. Li, X. Jia, and R. Lioutikov, "Goal-Conditioned Imitation Learning using Score-based Diffusion Policies," in *Proceedings of Robotics: Science and Systems*, DOI 10.15607/RSS.2023.XIX.028, Daegu, Republic of Korea, Jul. 2023.

[6] W. Si, N. Wang, R. Harris, and C. Yang, "Deep multimodal imitation learning-based framework for robot-assisted medical examination," *IEEE Transactions on Industrial Electronics*, vol. 73, DOI 10.1109/TIE.2025.3589442, no. 1, pp. 928–936, 2026.

[7] Y. Wang, H. Chen, C. Xu, and S. Cai, "An intuitive multi-task manipulation policy for imitation learning in simulated and real robots," *IEEE Transactions on Industrial Electronics*, vol. 73, DOI 10.1109/TIE.2026.3658676, no. 7, pp. 10 254–10 266, 2026.

[8] S. Zhong, X. Wang, X. Wang, H. Diao, G. Wang, K. C. Teh, Y. Shi, and B. Peng, "Information potential: A novel guidance framework for diffusion reinforcement learning," *IEEE Transactions on Industrial Electronics*, DOI 10.1109/TIE.2026.3699885, pp. 1–12, 2026.

[9] M. Du and S. Song, "Dynaguide: Steering diffusion polices with active dynamic guidance," in *Advances in Neural Information Processing Systems*, D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, Eds., vol. 38, Main Conference, DOI 10.52202/085713-1473, pp. 44 192–44 221. Curran Associates, Inc., 2025. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/file/3ec6c6fc9065aa57785eb05dffe7c3db-Paper-Conference.pdf

[10] Z. Zhou, A. Peng, C. Xu, Q. Li, T. Springenberg, K. Frans, and S. Levine, "Test-time gradient guidance of flow policies in reinforcement learning," *arXiv preprint arXiv:2606.11087*, 2026.

[11] C. Zhou, C. Hou, J. Hu, F. Luan, Z. Wang, Y. Dong, Y. Zhou, and B. He, "Language-driven bimanual cloth manipulation with llm planning and part-aware perception," *IEEE Transactions on Industrial Electronics*, DOI 10.1109/TIE.2026.3732441, pp. 1–13, 2026.

[12] Z. Wang, D. K. Jha, A. H. Qureshi, and D. Romeres, "Ppguide: Steering diffusion policies with performance predictive guidance," in *2026 IEEE*

International Conference on Robotics and Automation (ICRA), DOI 10.1109/ICRA57385.2026.11696698, pp. 18 339–18 346, 2026.

[13] A. Tang, W. Chen, A. Wagenmaker, C. Finn, and S. Levine, "Improving robotic generalist policies via flow reversal steering," *arXiv preprint arXiv:2606.13675*, 2026.

[14] Y. Wang, L. Wang, Y. Du, B. Sundaralingam, X. Yang, Y.-W. Chao, C. Pérez-D'Arpino, D. Fox, and J. Shah, "Inference-time policy steering through human interactions," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, DOI 10.1109/ICRA55743.2025.11127931, pp. 15 626–15 633, 2025.

[15] H. Ren, Z. Bi, Y. Zeng, L. Zheng, Z. Li, Z. Wan, L. Qi, and H. Cheng, "Fisher-preserving guidance: Training-free manifold constraints for safe diffusion control," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, T. Zhang, M. Dudik, M. Jaggi, A. Agarwal, S. Li, D. Schuurmans, J. Zhu, F. Berkenkamp, H. Dong, and A. Bietti, Eds., vol. 306, pp. 104 364–104 390. PMLR, 06–11 Jul 2026.

[16] H. Zhong, X. Zheng, Y. Sun, and J. Huang, "Task-dp3: Goal-centric perception-driven adaptive trajectory generation for robotic manipulation," *IEEE Transactions on Industrial Electronics*, vol. 73, DOI 10.1109/TIE.2025.3647869, no. 5, pp. 7392–7401, 2026.

[17] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, "3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations," in *Proceedings of Robotics: Science and Systems*, DOI 10.15607/RSS.2024.XX.067, Delft, Netherlands, Jul. 2024.

[18] H. Ding, N. Jaquier, J. Peters, and L. Rozo, "Fast and robust visuomotor riemannian flow matching policy," *IEEE Transactions on Robotics*, vol. 41, DOI 10.1109/TRO.2025.3601293, pp. 5327–5343, 2025.

[19] Q. Zhang, Z. Liu, H. Fan, G. Liu, B. Zeng, and S. Liu, "Flowpolicy: Enabling fast and robust 3d flow-based policy via consistency flow matching for robot manipulation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 14, pp. 14 754–14 762, 2025.

[20] T. Yoneda, L. Sun, G. Yang, B. C. Stadie, and M. R. Walter, "To the noise and back: Diffusion for shared autonomy," in *Proceedings of Robotics: Science and Systems*, DOI 10.15607/RSS.2023.XIX.014, Daegu, Republic of Korea, Jul. 2023. [Online]. Available: https://roboticsproceedings.org/rss19/p014.html

[21] L. Sun, J. Ji, H. Chen, J. Zhou, and M. R. Walter, "Flow-based policy adaptation without policy updates," *arXiv preprint arXiv:2606.06461*, 2026.

[22] J. Kim, B. S. Kim, and J. C. Ye, "Flowdps: Flow-driven posterior sampling for inverse problems," in *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 12 328–12 337. IEEE, 2025.

[23] J. Park and J. C. Ye, "Flowlps: Langevin-proximal sampling for flow-based inverse problem solvers," *arXiv preprint arXiv:2512.07150*, 2025.

[24] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *International Conference on Learning Representations*, 2023.

[25] X. Liu, C. Gong, and Q. Liu, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in *International Conference on Learning Representations*, 2023.

[26] S. L. Cotter, G. O. Roberts, A. M. Stuart, and D. White, "Mcmc methods for functions: Modifying old algorithms to make them faster," *Statistical Science*, vol. 28, no. 3, pp. 424–446, 2013. [Online]. Available: http://www.jstor.org/stable/43288425

[27] N. Parikh and S. Boyd, "Proximal algorithms," *Foundations and Trends in Optimization*, vol. 1, DOI 10.1561/2400000003, no. 3, pp. 127–239, 2014.

[28] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4rl: Datasets for deep data-driven reinforcement learning," *arXiv preprint arXiv:2004.07219*, 2020.

[29] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu, "robosuite: A modular simulation framework and benchmark for robot learning," *arXiv preprint arXiv:2009.12293*, 2020.

[30] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine, "Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning," in *Conference on robot learning*, pp. 1094–1100. PMLR, 2020.

Chunxiao Lu is currently working toward the Ph.D. degree in mechanical engineering from the Harbin Institute of Technology, Harbin, China. Her research interests include imitation learning, generative models, and robotics.

Yongyao Li

Jun Li received his B.Eng. degree in automotive engineering from the Harbin Institute of Technology, Weihai, China, in 2015, and his M.Eng. degree in mechanical engineering from the Harbin Institute of Technology, Harbin, China, in 2017. From 2021 to 2022, he was a visiting Ph.D. student at the University of Leeds, Leeds, UK. He is currently working toward a Ph.D. degree with the Harbin Institute of Technology. His research interests include optimization-based planning and control of legged robots

Yufei Liu

Weihua Li (Member, IEEE) received the B.S., M.S. and Ph.D. degrees in Manufacturing Engineering of Aerospace Vehicle from Harbin Institute of Technology, Harbin, China, in 2009, 2011 and 2016, respectively. From 2013 to 2015, he was a visiting researcher with the University of Alberta, Edmonton, Canada. He is currently a professor with the School of Automotive Engineering in Harbin Institute of Technology (Weihai). His research interests include the teleoperation control of mobile robot, the dynamic simulation and teleoperation of planetary rover.